ComicEngine: A Code-First Pipeline for Consistent, Cost-Aware AI History Comics

Technical Report on the *onceuponatime* System

Living Phase Log (updated after each phase gate)

Snapshot: 9 August 2026 · Phases 0–11 + Romance pipeline HIMYM Ep1 Steps 1–5 complete

Avinash Nandyala
Independent Project / ComicMainEngine
`github.com/navinash47/ComicMainEngine`

Report regenerate: `python scripts/build_report.py`

Abstract

We present **ComicEngine**, a modular, code-first system for generating educational history comics with recurring characters, tracked API spend, and human-in-the-loop curation. Built for *onceuponatime* (bedtime narratives that explain real events to children), the system separates *coding/LLM* traffic (optionally via OmniRoute at `localhost:20128`) from *image generation* (direct commercial APIs: fal FLUX, Google Gemini image). This living report records what each phase actually ran, what worked, quantitative ledger totals from SQLite (`data/usage.db`), style bake-off judge scores, and interim character-consistency findings. **As of this snapshot:** Phase 9 adds an ROI dashboard at `/roi` with unit economics ($/story, $/panel, 100-episode projection), spend breakdowns by phase/provider/category/purpose, charts, and model recommendations from the live ledger (≈$2.96 R&D so far).

1 Introduction

Production AI comics need character/style consistency, predictable cost, commercially licensed image paths, and an auditable spend ledger. ComicEngine addresses these with phasal CLIs, Pydantic episode JSON, style presets, and a FastAPI dashboard (Tasks, Stories, Analytics, Images, and this Report PDF at `/report`).

Test story. End-to-end stress test: dramatized bedtime episode about the Cockroach Janta Party (CJP) / fair-exam student protests (characters including Abhijeet Dipke, Saurav Das, public figures portrayed respectfully for pipeline testing only).

Living-document policy. After every phase gate we (1) append empirical results to this `.tex`, (2) rebuild the PDF, and (3) include the report update in the same phase merge to `master`.

2 Routing and Licensing Constraints

- **Text / coding LLMs:** OmniRoute when `USE_OMNIROUTE=1` + `OMNIROUTE_API_KEY`.
- **All images:** direct `.env` keys only (`FAL_API_KEY`/`FAL_KEY`, `GOOGLE_API_KEY`) — never OmniRoute.
- Local FLUX `[dev]` / Kontext `[dev]` remain R&D-only under BFL non-commercial terms; production-like finals use commercial fal/Gemini APIs.

3 Architecture (stable through Phase 3)

`src/comicengine/`: config, pricing, tracked clients, styles, analytics, stories, gallery, tasks. `scripts/`: phase CLIs. `dashboard/`: FastAPI + static UI. `data/usage.db`: per-call ledger. `data/style_lock.json`: bake-off + human-judge flags. `outputs/`: episode JSON, images, reports.

Evaluation stack. Pillow heuristics (existence, min dims, mean brightness/variance anti-blank) write `image_quality` into call metadata. Analytics derives model recommendations (`data/model_recommendations.json`) — advisory only; scripts do not auto-switch.

4 Cumulative Ledger Snapshot

Source: live SQLite summary at report build time.

Unit costs that worked in practice. fal FLUX. `schnell`: ≈ $0.003/image, ∼1.5–2.4 s. Gemini `gemini-3.1-flash-image-preview`: ≈ $0.039/image, ∼9–12 s. fal FLUX. `pro/kontext`: ≈ $0.040/image, ∼11–14 s. Text script generation (Claude Haiku): ≈ $0.046 for the CJP episode.

Table 1: Spend and success by phase totals (ledger).

Phase	Calls	Cost (USD)	Notes
phase0	13	0.003	provider pings
phase0.5	2	0.046	CJP script
phase1	1	0.003	fal schnell proof
phase2	13	0.093	style bake-off + judge
phase3	23	0.387	refs + consistency + judge
phase4	11	0.655	Caesar + Hitler scripts
phase5	66	1.772	refs + 44 panels
total	129	**2.960**	incl. retries

Table 2: Top providers by estimated spend (post Phase 3).

Provider	Calls	Cost (USD)
google	10	0.273
fal	28	0.158
omniroute:anthropic	4	0.053
anthropic (direct)	4	0.046
omniroute:openai	3	0.002

5 Phase Log — What Ran and What Worked

5.1 Phase 0 — Environments & pings

Goal. Authenticate providers; prove routing. **What worked.** Anthropic Claude Haiku, OpenAI `gpt-4.1-mini`, OmniRoute `auto/cheap` for Anthropic and OpenAI, Gemini text via **direct** Google. **What failed / adapted.** Older Gemini Flash IDs error under "new user" model restrictions; replaced with `gemini-3.1-flash-lite` (text) and later `gemini-3.1-flash-image-preview` (image). One logged text error remains in the ledger (Gemini model mismatch during early ping). **Gate.** Success when calls land in SQLite with status `ok`.

5.2 Phase 0.5 — Script engine

Goal. Structured bedtime episode JSON for Stories UI. **What worked.** Claude Haiku → Pydantic episode `outputs/phase0.5/episode_cjp_origin.json`: title *The Cockroach Janta Party: When Students Ask for Fair Exams*, **12 panels**, fact sheet + `art_prompts`. Wikimedia Commons location refs fetched into `outputs/phase0.5/refs/` with attribution manifest (place/atmosphere refs, not official CJP press kits). **Cost.** ≈$0.046 text (2 Anthropic calls in phase tag). **Evaluation.** Human-readable in dashboard Stories; suitable as Phase 1–3 prompt source.

5.3 Phase 1 — Single commercial image path

Goal. Prove fal FLUX `schnell` with tracked image category + QA. **What worked.** `scripts/phase1_generate.py -backend fal` → `outputs/phase1/flux_schnell_fal.png`; QA `pass`; $0.003. **Evaluation.** Establishes the cheap default image backend for bake-offs.

Table 3: LLM auto-judge rubric totals (higher better; max domain score 10 each).

Style	Imm.	Bed.	Civic	Cost	Total
painterly_storybook	7	10	8	9	**34**
graphic_novel	8	7	10	9	**34**
watercolor_editorial	7	8	9	9	33
korean_manhwa	9	6	7	9	31

5.4 Phase 2 — Style bake-off (complete; human lock pending)

Setup. Four presets × two fixed beats (Abhijeet+students; dad–daughter bedtime) via fal `schnell`: `korean_manhwa`, `painterly_storybook`, `graphic_novel`, `watercolor_editorial`. Eight bake-off images under `outputs/phase2/bakeoff/`; all QA scores **1.0 / pass**. Total image spend for 8 equal-cost styles: **$0.024**.

Auto-judge conclusion. Winner: `painterly_storybook` (bedtime safety). Closest civic alternative: `graphic_novel`. Cheap good-enough: `watercolor_editorial`. Manhwa won immersion (9) but scored lowest bedtime safety (6).

Human policy (current). `human_judge_pending=true`; `preferred_while_judging=korean_manhwa` so active rendering stays webtoon-like while the operator compares finals in the Images tab. Override anytime with `STYLE_ID` or by editing `data/style_lock.json`.

Phase 2 also generated. Style grid previews and Gemini comparison sample (`style_gemini_01_...`); Gemini image is ≈13× fal `schnell` unit cost.

5.5 Phase 3 — Character consistency (complete)

Goal. Expanded cast reference sheets + method bake-off for identity lock across poses. **Artifacts.** `outputs/phase3/consistency_manifest.json`, `decision.json`; code: `scripts/phase3_consistency_bakeoff.py`, `characters_phase3.py`, client helpers `fal_kontext_edit` / `gemini_image_with_refs`. Merged to `master` as commit `5078ead`.

What worked (22/22, 0 failures).

- **8 ref sheets** (fal `schnell`): Abhijeet, Saurav Das, Modi, Amit Shah, Sonam Wangchuk, police, dad, daughter.

Table 4: Phase 3 method judge scores (higher better).

Method	Face	Outfit	Style	Rel.	Cost	Total
`gemini_ref`	8	8	8	9	7	**40**
`flux_kontext`	9	9	8	7	3	36
`text_only_fal`	3	3	7	9	10	32

- **14 consistency scenes** under interim style `korean_manhwa`: Abhijeet ×2 scenes ×3 methods; Dad/Modi ×2 scenes ×2 methods (`text_only_fal`, `gemini_ref`).
- All Pillow QA verdicts `pass`; ≈$0.387 phase spend (images direct fal/Google; OmniRoute used only for text judge).

Decision (ship). Winner: **`gemini_ref`** (Gemini Flash Image + reference sheet) as default identity lock for hero cast. Reserve **`flux_kontext`** for hard Abhijeet closeup/speak retries when face drift appears. Use **`text_only_fal`** only for cheap drafts/extras (weak cross-scene identity despite perfect blank-frame QA).

Evaluation caveats (from decision notes). Local QA checks blanks/dims/brightness—not identity; Kontext under-sampled (n=2, Abhijeet-only); Gemini aspect ratios sometimes vary (e.g. 848×1264). Human eye review on Images tab remains recommended before episode mass generation.

5.6 Phase 4 — Multi-topic scripts (complete for gate)

Goal. Generalize Topic→episode JSON beyond CJP; ship two hard-history episodes written *through character lenses* with modern-affairs analogies (teen/young-adult didactic, user-selected framing).

What worked.

- Character bibles: `characters_caesar.py`, `characters_hitler.py` (ideology/thinking in `notes`).
- Generalized `generate_episode(story_key)` + registry `STORIES` (`cjp`, `et_tu_brutus`, `hitler_warning`).
- CLI `scripts/phase4_scripts.py`; OmniRoute default model `auto` (`auto/cheap` often skipped `tool_use`).
- Robustness: retries, JSON-only fallback, normalize panels (speaker lists → dialogue strings; verdict aliases).

Stories now in dashboard (beside CJP).

- `episode_cjp_origin` (phase0.5, 12 panels) — fair exams / youth protest.
- `episode_et_tu_brutus` (phase4, 16) — Caesar, Brutus, Cassius, Antony, People; analogies: fragile republic, personality cult, assassination is not a liberty reset.

Table 5: Phase 5 full-batch results (all QA pass).

Story	Panels ok	Errors	Primary meth
episode_cjp_origin	12/12	0	gemini_ref
episode_et_tu_brutus	16/16	0	gemini_ref
episode_hitler_warning	16/16	0	gemini_ref ×11 + kontext
total	**44/44**	**0**	Phase spend ≈$

- `episode_hitler_warning` (phase4, 16) — Hitler/Goebbels/Eichmann ideology *exposed then condemned*; Jewish teen + resistance + bystander; analogies: propaganda, minority hate, bureaucracy of atrocity, legal demolition of democracy.

Cost. Phase 4 script spend ≈$0.655 (11 text calls incl. retries). Cumulative ledger ≈$1.19.

Ethics evaluation. Hitler episode is explicitly anti-fascist education (no gore; no Nazi glamour). Character POV is used so propaganda patterns are recognizable; Dad/victims close the frame. Requires human fact-check before any public publish.

5.7 Phase 5 — Panel batch generator (complete)

Goal. Episode JSON → panel PNGs with Phase 3 consistency defaults + retries + cost ledger.

What worked.

- `comicengine.panel_batch` + `scripts/phase5_panel_batch.py`
- Ref sheets under `outputs/phase5/refs/` (reuse Phase 3 when present; else fal `schnell`)
- Default render: `gemini_ref` (up to 3 cast refs) → QA-fail/error falls back to `flux_kontext` then `text_only_fal`
- Writes `image_path/image_method/image_qa` into episode JSON; Stories UI shows `/media/...` images

Evaluation. Perfect pipeline success rate for this batch. Kontext retries concentrated on Hitler-era panels (likely safety/policy or identity hardness) — validates the Phase 3 ship rule: gemini default, Kontext for hard retries. Human visual review still required for identity drift and historical care (especially atrocity education frames).

5.8 Phase 6 — Speech bubble compositor + story reader (complete)

Goal. Readable on-image dialogue/captions and a webpage that feels like reading a comic (not only a dump of PNGs).

Table 6: Phase 7 assembly outputs.

Story	Panels assembled	Artifacts
episode_cjp_origin	12	webtoon + PDF
episode_et_tu_brutus	16	webtoon + PDF
episode_hitler_warning	16	webtoon + PDF

What worked.

- Pillow compositor (`comicengine.compositor`): rounded speech bubbles (speaker labels + wrapped lines), caption bar, local Arial font.
- CLI `scripts/phase6_compose.py` wrote `outputs/phase6/<story>/panel_XX.png` and `composed_image_path` on each panel.
- **44/44** panels composed, 0 errors (offline, no API cost).
- Stories UI: cast list, fact sheet intro, "Opening/Meanwhile" bridge narration between panels, composed images via `/media/...`, dialogue + caption text beneath each panel, closing fact checks.

5.9 Phase 7 — Episode assembler (complete)

Goal. One readable artifact per episode: long vertical webtoon + printable PDF.

What worked.

- `comicengine.assembler` + `scripts/phase7_assemble.py`
- Cover strip + composed Phase 6 panels stacked at width 900 with gutter; multi-page PDF via Pillow.
- Outputs: `outputs/phase7/<story>/webtoon.png`, `episode.pdf`, `manifest.json`
- Stories pages expose webtoon preview + PDF/webtoon buttons.

5.10 Phase 7.5 — Only Image + Story Library (complete)

Goal. Store UI stories centrally; also ship bubble-free visual editions.

What worked.

- `edition='image_only'` assembler path → `outputs/phase7.5/<story>/webtoon_image_only.png` + `episode_image_only.pdf`
- CLI `scripts/phase7_5_image_only.py` for all three stories
- `comicengine.library` catalog at `outputs/library/catalog.json`
- Dashboard `/library` with Reader vs Only-Image editions; nav link sitewide

5.11 Phase 8 — Curation CLI + SQLite (complete)

Goal. Human gate: approve / reject / regenerate panels and episodes with durable status.

What worked.

- SQLite table `curation_item` in `data/usage.db` via `comicengine.curation`
- CLI `scripts/phase8_curate.py` (`seed|list|summary|approve|reject|regenerate`)
- Seeded 47 rows (3 episodes + 44 panels)
- Library UI chips + episode approve/reject; Alt-click regenerates panel (re-render + recompose)
- APIs: `/api/curation`, `/api/curation/seed`, `/api/curation/{id}`, regenerate endpoint

5.12 Phase 8.5 — Panel rating + prompt editor (complete)

Goal. Every reject/regen opens an editable art prompt; panels carry a human rating and free-text suggestions so quality is visible on the site and refreshes after regen.

What worked.

- Extended `curation_item` with `rating` (1–5) and `suggestions`
- `panel_editor_payload` + `regenerate_panel(..., prompt=, rating=, suggestions=)`; phase tagged `phase8.5`; edited `art_prompt` persisted into episode JSON
- `render_panel(..., prompt_override=)` uses the user-edited scene text
- Library + Story reader modal: stars, suggestions, prompt textarea; reject/regen refresh images via cache-busting `?v=`
- CLI `scripts/phase8_5_panel_editor.py`; APIs for panel editor GET and JSON regenerate body

5.13 Phase 8.6 — Reader auth + dual Vercel hosting (complete)

Goal. Separable reader vs admin surfaces; reviewers rate panels/stories without seeing engine stats; ship a shareable beta without putting image regen on Vercel.

What shipped.

- Local FastAPI: open owner console (`/admin`, `/stats`); public comics review at `/review` with thumbnail cards, Read&rate / PDF / Just read, and Mom Test after ratings; Reader CRM at `/reviewers`

- Vercel **comicengine-reader** (`https://comicengine-reader.vercel.app`): username/password (scrypt + HttpOnly `ce_session` cookie + Upstash Redis)
- Library cards: evaluated 3:2 thumbnails, title, three actions — **Read & rate** / **PDF** / **Just read**
- Thumbnail picks (`scripts/select_thumbnails.py`): CJP panel 5, Et tu panel 11, Hitler-warning panel 1 (visual override away from spectacle beats)
- Vercel **comicengine-admin** (`https://comicengine-admin.vercel.app`): password gate; snapshot spend/ROI/tasks + **live** Upstash people/logins/feedback + canvas charts (20 s poll)
- Branding: **ComicEngine** + **Beta** pill; footer © 2026 Avinash Nandyala (reader, admin, local dashboard)
- Hosting runbook `docs/HOSTING.md`; shared Upstash resource on both projects
- After panel/story ratings: 14-question Mom Test questionnaire (Upstash); answers shown per person in admin live console

Ledger at gate (snapshot). Engine usage snapshot still $2.96 / 138 calls (dominant spend remains Phase 5 image batch). Hosted reader/admin traffic is Upstash + static Vercel — not counted as OmniRoute/image ledger cost.

5.14 Phase 9 — ROI dashboard (complete)

Goal. Make spend actionable: unit economics + charts from the SQLite ledger.

What worked.

- `comicengine.roi.roi_dashboard()` aggregates category/purpose/daily spend + $/story and $/panel
- Dashboard page `/roi` + `/api/roi`; nav link sitewide
- Snapshot JSON `outputs/phase9/roi_snapshot.json` via `scripts/phase9_roi.py`

5.15 Phase 10 — Publishing pipeline (complete)

Goal. Export approved Library editions for Cloudflare Pages + R2; keep live auth/feedback on FastAPI.

What worked.

- `comicengine.publish.export_beta_site()` → `outputs/phase10/site`
- CLI `scripts/phase10_publish.py` (`export|upload-r2|deploy-pages|gate`)
- Live Google login remains on the FastAPI host (Tunnel / public HTTPS); Pages is the static mirror

5.16 Phase 11 — Cost guardrails (complete)

Goal. Freeze beta spend rules before Version 2; defer expensive cheaper-backend bake-offs until Mom Test priorities are clear.

What worked.

- Snapshot `outputs/phase11/cost_guardrails.json` with ROI baseline + reviewer/admin regen policy
- Explicit roadmap: **Version 2 begins at Phase 12**, feature work driven by exported Mom Test feedback

5.17 Romance pipeline — How I Met Your Mother Ep1 Infatuation (Steps 1–5 complete)

Goal. Dad→daughter love-story episode (korean_manhwa) with Maya teasing cuts and protagonist micro-feelings (butterflies, electricity, hand-hover, Limbo Date, feather).

Steps (TaskObserver stepper).

1. `phase_romance_step1` — script locked (**76 panels**) at `outputs/phase4/episode_how_i_met_your_mother_ep1.json`
2. `phase_romance_step2` — manhwa character refs (8/8 cast sheets)
3. `phase_romance_step3` — panel batch (`STYLE_ID=korean_manhwa`, mostly `gemini_ref`): **76/76 ok**
4. `phase_romance_step4` — speech bubbles / captions composed (76/76)
5. `phase_romance_step5` — webtoon + PDF + Library (`outputs/phase7/.../webtoon.png`, image-only under phase7.5)

Dashboard. Home Romance pipeline stepper + live ledger line (total spend + romance-phase spend) polled every 2s with KPIs.

Story. Father tells daughter how he met her mother (Rohan / Elena / Meridian Soft Bay trip).

Cost gate. Pre-romance ledger ≈$2.96 / 138 calls. After Steps 1–5: romance pipeline spend ≈$2.88 (refs $0.018 + panels $2.86; compose/assemble local $0); full ledger ≈$5.84 / 223 calls.

6 Dashboard Surfaces

Local (`http://127.0.0.1:8765`): usage KPIs, TaskObserver, Stories, Library, ROI, Analytics, Images, admin console, public `/review`, Report PDF (`/report`).

Hosted beta: reader `https://comicengine-reader.vercel.app`; fellowdev admin `https://comicengine-admin.vercel.app` (password-gated). Live image regen / OmniRoute scripting remain on the laptop (or a GPU/API host).

7 What We Explicitly Learned

1. OmniRoute is excellent for LLM scripting/judging; keeping images off OmniRoute avoided opaque image billing and matched commercial-key policy.
2. fal `schnell` makes wide style exploration financially trivial (~$0.024 for 8 bake-off frames).
3. Auto style judges trade off immersion vs. bedtime safety; human adjudication remains mandatory for product voice.
4. Pillow "pass" is necessary but not sufficient for character consistency; the Phase 3 LLM judge correctly ranked `gemini_ref` over cheaper text-only despite identical QA scores.
5. OmniRoute model routing matters: prefer `auto` (or provider-prefixed Claude) when `auto/cheap` returns thinking text without `tool_use`.
6. Vercel serverless session UX needs HttpOnly cookies (Bearer-only + aggressive `/me` clears caused login loops); CSS `display` on `.btn` overrides HTML `[hidden]`.
7. Admin canvas charts must lock CSS height — reading `canvas.height` after DPR resize compounds every live poll.

8 Risks and Ethics

CJP bedtime lane remains ages ∼7–10; Phase 4 hard-history lane is teen didactic (∼13–17). Hitler content must never romanticize Nazism; Caesar content must not romanticize political murder. Respectful depiction of public figures; Commons attribution for place photos; human fact-check before any public publish. Style and consistency locks must not be decided solely by LLM judges or blank-frame QA. Public beta URLs expose comics widely — keep admin password private; Upstash holds reader hashes/sessions/feedback.

9 Next Gates

- Human style lock: clear `human_judge_pending` (manhwa vs painterly).
- Collect panel/story ratings from beta readers; harvest admin live people analytics for Version 2 priorities.
- Re-run `prepare_admin_site.py` (+ redeploy) whenever ledger/ROI snapshot must refresh.
- Version 2 / Phase 12+: feature work driven by real reader feedback (not vanity NPS).

10 Conclusion

ComicEngine now covers script→render→compose→assemble→curate→ROI→hosted reader/admin beta→publish packaging. Version 2 starts at Phase 12 using logged panel/story feedback from beta readers.

A Reproduce

```
source .venv/bin/activate
pip install -e .
# configure .env (never commit)
python scripts/run_dashboard.py
python scripts/build_report.py
# PDF: http://127.0.0.1:8765/report
```

B Changelog (report editions)

2026-08-09 a
Initial two-column architecture draft.

2026-08-09 b
Living phase log: ledger tables, Phase 2 judge matrix, Phase 3 interim notes.

2026-08-09 c
Phase 3 gate closed: 22/22 images, winner `gemini_ref` (judge 40), ledger $0.532; dashboard `/report` link.

2026-08-09 d
Phase 4 scripts: Et tu Brutus + Hitler warning (teen didactic, character POV) beside CJP; ledger ≈$1.19.

2026-08-09 e
Phase 5: 44/44 panels rendered (`gemini_ref`/Kontext); Stories show images; ledger ≈$2.96.

2026-08-09 f
Phase 6: speech bubbles/captions on 44 panels; reader bridges + cast/facts on Stories pages.

2026-08-09 g
Phase 7: vertical webtoon + PDF for all three stories; Stories download/preview links.

2026-08-09 h
Phase 7.5: Only Image editions + Story Library (`/library`) catalog.

2026-08-09 i
Phase 8: curation SQLite + CLI + Library approve/reject/regenerate.

2026-08-09 j
Phase 9: ROI dashboard (`/roi`) unit economics + spend charts.

2026-08-09 k
Phase 8.5: panel rating + editable art prompt on reject/regen.

2026-08-09 l
Phase 8.6: dual Vercel reader/admin + Upstash auth; thumbnails; live people analytics; Beta branding; ledger snapshot still ≈$2.96 / 138 calls.